Solana's Confidential Transfer Audit -Final report

Intermediary Audit Report can be seen in

Background

Solana's Token-2022 Program, through the Confidential Transfer extensions, enables users to transact with encrypted balance amounts while preserving blockchain transaction verifiability through appropriate cryptographic schemes. Specifically, the extension employs twisted ElGamal encryption, sigma protocols for proving algebraic relationships, and Bulletproof-based range proofs to establish transaction validity without revealing actual amounts.

The CT extensions supports a dual balance architecture where each account can simultaneously hold both a confidential balance and a non-confidential balance. Users can deposit funds from their public balance into the confidential balance, perform private transfers between confidential balances, and withdraw back to the public balance when needed.
This separation enables efficient homomorphic updates to encrypted balances while allowing recipients to decrypt and reconcile their funds asynchronously.

Optional auditor keys can be configured at the mint level, allowing designated parties to decrypt transfer amounts for compliance purposes without affecting the privacy guarantees for other observers.

Overall Audit Objectives

The primary objectives of the overall audit are to:

- Review all three Confidential transfer extensions.
- Review the associated documentation provided by the client.
- Assess the security of the theoretical protocol and sub protocols.
- Review the interaction of the Confidential transfer extensions with token-2022 base functionality.
- Review the interaction of the Confidential transfer extensions with other extensions.
- Audit the zk ElGamal proof program

- Audit the implementation of the sigma proofs for completeness, computational knowledge soundness and for the zero-knowledge properties
- Audit the implementation of various range proofs for soundness
- Audit the implementation of the twisted ElGamal encryption scheme and its integration with various proof systems
- Test edge cases and potential security bypass scenarios
- Provide recommendations for additional security hardening

The Scope of the Final Report

The audit's top priority is the verification path of the CT protocol and the verification path in `zk-elgamal-proof` repository.

The detailed scope of the final report is listed below:

- The Interaction of CT with token-2022 functionality
  - `program/src/processor.rs` as a starting point for CT interactions
- The correctness of all three CT extensions
  - Emphasis on verification and soundness
  - Transaction generation and auxiliary code were also reviewed
  - All Code in all three CT extensions folders:
    - `program/src/extension/confidential_transfer/`
    - `program/src/extension/confidential_transfer_fee/`
    - `program/src/extension/confidential_mint_burn/`
  - Ciphertext operations: `confidential/ciphertext-arithmetic/`
  - ZK ElGamal Proof usage: `confidential/proof-extraction/src/`
  - ElGamal Registry `program/confidential/elgamal-registry` (partial review)
- The ZK ElGamal proof program
  - Emphasis on verification and soundness
  - Transaction generation was also reviewed
  - As reported in the intermediary report and continued in this report

Methodology

Our audit approach combines:

- **Documentation Review:** Comprehensive analysis and review of protocol specifications and cryptographic design documents as provided by the client
- **Code Audit:** Line-by-line audit of implementation code with focus on security-critical sections

    - **Cryptographic Analysis:** Verification of the completeness, soundness an of the zero-knowledge properties for the implementations of sigma protocols. Verification of the soundness of various range proofs and of the implementation of the Confidential Transfer algorithms
    - **Attack Vector Testing:** Reproduction and testing of known attack patterns and novel exploit scenarios
- **Test Coverage Analysis:** Evaluation of existing test suites and creation of additional test cases

Team

This audit is being conducted by:

- Pablo Kogan
- Oana Ciobotaru
- Constance Beguier
- Arseni Kalma

Key Resources

Our audit relies on the following resources:

- The token-2022 repository: https://github.com/solana-program/token-2022
    - Audited commit: https://github.com/solana-program/token-2022/tree/27df8bd63d4808971a806332f706ea6b4de577b8
- Official Agave repository: https://github.com/anza-xyz/agave
    - Audited commit: https://github.com/anza-xyz/agave/commit/3981b9334d65f496ae056644215f4519a9bfa8a0
- The zk-elgamal-proof repository: https://github.com/solana-program/zk-elgamal-proof
    - Audited commit: https://github.com/solana-program/zk-elgamal-proof/commit/8c84822593d393c2305eea917fdffd1ec2525aa7
- Previous vulnerability analysis:
    - Post Mortem: ZK ElGamal Proof Program Bug, June 2025
    - Uncovering the Phantom Challenge Soundness Bug in Solana's ZK ElGamal Proof Program
- Additional resources as provided by the Anza team
- Description of the sigma protocols used

Audit time-frame

From 28th July 2025 until 19 Dec 2025

Overall Impressions

The Confidential Transfer (CT) codebase is well-written and maintains good readability throughout. It is also well tested across all abstraction layers.

Aside from the standalone correctness issues presented in this report, the high-level concerns that we see affecting CT correctness and Token-2022 maintainability long-term are:

1. **Interaction between Token-2022 extensions** - All CT functionality is implemented as extensions to Token-2022. In addition to the three CT extensions there are 13 other extensions. The extension mechanism is manual, with dispatch logic primarily encapsulated in `program/src/processor.rs`. The most significant advantage of this approach is explicit control flow. Everything is visible and can be inspected without hidden assumptions.

   However, since each extension has the potential to affect the core Token-2022 operations (`process_transfer`, `process_approve`, `process_mint_to`, etc.) and can potentially interact with every other extension's logic, this manual style of conditional checking does not scale well, is error-prone, and will become increasingly difficult to maintain over time.

   Some of our most significant findings are related to interactions between extensions.

2. ZK-Elgamal-proof program impressions, as reported in the intermediately report.

To address these issues we propose a set of concrete recommendations:

Findings

Mathematical Correctness

We have reviewed the following documents and we attach below the recommended edits and corrections:

- Twisted ElGamal description:

> twisted_elgamal.pdf

For the following documents we have reviewed Section 3 of the respective write-ups and we attach below the recommended corrections:

- Public key validity argument:

> pubkey_proof.pdf

- Grouped ciphertext validity argument

> ciphertext_validity.pdf

- Ciphertext ciphertext equality argument

> ciphertext_ciphertext_equality.pdf

- Ciphertext commitment equality argument

ciphertext_commitment_equality.pdf

- Percentage with cap argument **- as discussed with the Anza team**

percentage_with_cap.pdf

- Zero ciphertext validity argument - all clear.

Additionally, we have created the specification for the batched grouped ciphertext validity proof as it appears in the zk-sdk implementation and we append it as described below:

Batched_grouped_ciphertext_validity.pdf

We were not provided with a security proof for the batched grouped ciphertext validity as it appears in the zk-sdk implementation so we wrote our own as per below:

Proof_batched_grouped_ciphertext_validity.pdf

Finally, in order to audit the range proof based code, we have also read and mapped the theory from the Bulletproofs paper Section 3, Sections 4.1 - 4.3 and Section 6.2 to the corresponding code, especially regarding the mod.rs and inner_product.rs implementation.

For code issues, we define 4 severity levels

- Critical - 2 total
- High impact - 1 total
- Medium impact - 11 total
- Low impact / best practices recommendation - 4 total

The Confidential Transfer extension

Issue: Permanent Delegate Bypass via Confidential Transfers [#b01]

As discussed with the Anza team

Severity: Critical

Description:

Confidential transfers never check permanent delegate. As a result:

- Users can evade permanent delegate control by moving to confidential balance.
- Permanent delegate cannot perform confidential transfers.

Real-world relevance: Regulated stablecoins (e.g., Paxos USDP) are configured with both `PermanentDelegate` and `ConfidentialTransfer` extensions. Users could evade authority actions by depositing to confidential balances. They are unaware of this gap.

Recommendations:

Two paths to mitigate:

1. (Easy) Disable co-existence: Reject mint initialization when both `PermanentDelegate` and `ConfidentialTransferMint` extensions are configured on the same mint

2. (Harder) Implement proper support: Add permanent delegate authority the ability to confidential transfer operations allowing the delegate to move funds or force-decrypt balances for regulatory compliance. Permanent delegate should have same power over confidential balances as over public balances.

To further clarify, in case no steps to mitigate are taken, the following scenario is possible:

1. User has 1000 USDP in `Account.amount` (public balance)

2. Issuer can Transfer via `PermanentDelegate` for regulatory compliance (e.g., court order, sanctions enforcement, AML seizure)

3. User calls `Deposit` instruction

4. Now user has 1000 in `ConfidentialTransferAccount.available_balance` (encrypted)

5. Issuer cannot enforce compliance - no instruction exists for permanent delegate to:

- Call `Transfer` on confidential balances (only checks owner, not permanent delegate)

- Call `Withdraw` (requires owner signature + ZK proof)

- Call `EmptyAccount` (requires owner signature)

Code:

file: `clients/rust-legacy/tests/ct_all_extentions.rs`

```
/// ConfidentialTransferMint and PermanentDelegate can coexist.
/// This demonstrates this setup is possible. However, PermanentDelegate cannot
/// access confidential balances.
#[tokio::test]
async fn test_confidential_transfer_with_permanent_delegate() {
    let authority = Keypair::new();
    let permanent_delegate = Keypair::new();

    let mut context = TestContext::new().await;
    context
        .init_token_with_mint(vec![
```

```
            ExtensionInitializationParams::PermanentDelegate {
                delegate: permanent_delegate.pubkey(),
            },
            ExtensionInitializationParams::ConfidentialTransferMint {
                authority: Some(authority.pubkey()),
                // Changing to false will not reduce the attack surface
                auto_approve_new_accounts: true,
                auditor_elgamal_pubkey: None,
            },
        ])
        .await
        .unwrap();

    let token_context = context.token_context.as_ref().unwrap();
    let token = &token_context.token;

    // Verify both extensions are present on mint
    let mint_state = token.get_mint_info().await.unwrap();
    let perm_delegate_ext = mint_state.get_extension::<PermanentDelegate>().unwrap
();
    assert_eq!(
        Option::<Pubkey>::from(perm_delegate_ext.delegate),
        Some(permanent_delegate.pubkey())
    );
    assert!(mint_state.get_extension::<ConfidentialTransferMint>().is_ok());

    // Create account, mint tokens, deposit to confidential balance
    let alice_meta = ConfidentialTokenAccountMeta::new_with_tokens(
        token,
        &token_context.alice,
        None,
        false,
        false,
        &token_context.mint_authority,
        1000,
        token_context.decimals,
    )
        .await;

    // Verify setup complete
    alice_meta
        .check_balances(
            token,
            ConfidentialTokenAccountBalances {
                pending_balance_lo: 0,
                pending_balance_hi: 0,
                available_balance: 1000,
```

```
                decryptable_available_balance: 1000,
            },
        )
        .await;

    // Permanent delegate cannot access confidential balances
}
```

Discussion:

We checked the Solana Paxos mint (USDP):

`HVbpJAQGNpkgBaYBZQBR1t7yFdvaYVp2vCQQfKKEN4tM`

From the official Paxos GitHub repository: paxosglobal/usdp-contracts and

https://solscan.io/token/HVbpJAQGNpkgBaYBZQBR1t7yFdvaYVp2vCQQfKKEN4tM

Using a dedicated program:

```
fn main() {
    let mint = Pubkey::from_str("HVbpJAQGNpkgBaYBZQBR1t7yFdvaYVp2vCQQfKKEN4tM").un
wrap();
    let client = RpcClient::new("https://api.mainnet-beta.solana.com");
    let account = client.get_account(&mint).expect("Failed to fetch");
    let state = StateWithExtensions::<Mint>::unpack(&account.data).unwrap();

    println!("Paxos USDP: {}\n", mint);
    println!("Extensions:");
    for ext in state.get_extension_types().unwrap() {
        println!("  - {:?}", ext);
    }

    if let Ok(e) = state.get_extension::<PermanentDelegate>() {
        println!("\nPermanentDelegate: {:?}", Option::<Pubkey>::from(e.delegate));
    }
    if let Ok(e) = state.get_extension::<ConfidentialTransferMint>() {
        println!("ConfidentialTransferMint: {:?}", Option::<Pubkey>::from(e.author
ity));
    }
}
```

and indeed:

```
Paxos USDP: HVbpJAQGNpkgBaYBZQBR1t7yFdvaYVp2vCQQfKKEN4tM

Extensions:
  - MintCloseAuthority
  - PermanentDelegate
  - ConfidentialTransferMint
  - TransferHook
```

```
  - MetadataPointer
  - TokenMetadata

PermanentDelegate: Some(2apBGMsS6ti9RyF5TwQTDswXBWskiJP2LD4cUEDqYJjk)
ConfidentialTransferMint: Some(2apBGMsS6ti9RyF5TwQTDswXBWskiJP2LD4cUEDqYJjk)
```

Or via GUI: https://solscan.io/token/HVbpJAQGNpkgBaYBZQBR1t7yFdvaYVp2vCQQfKKEN4tM#extensions

We believe that they are unaware of this issue and consider their stable-coin compliant.

Even if one of the mitigation paths is taken, the token has already been minted, so it is necessary to properly assess the required far-from-trivial mitigation in this case.

Also, it is critical to consider the existence of any additional issues / issuers.

Issue: CT `InitializeMint` Overwrite [#b02]

as discussed with the Anza team

Severity: High

Description:

The `ConfidentialTransferMint` extension can be maliciously overwritten when its initialization instructions are executed across multiple transactions.

`confidential_transfer::initialize_mint` **does not enforce any signature or authority check**, nor does it verify that the Confidential Transfer (CT) mint extension has not already been initialized. As a result, if an attacker submits their own confidential transfer `initialize_mint` instruction *after* the legitimate creator invokes it, but *before* the base SPL Token `initialize_mint` is executed, the attacker can overwrite all CT configuration fields (mint authority, auditor public key, auto-approve flag).

This enables a *silent privilege takeover* of the Confidential Transfer mint authority and auditor configuration:

- Attackers can set themselves as the Confidential Transfer mint authority.
- Attackers can replace the auditor ElGamal public key.

Although the official token-2022 client libraries typically bundle all mint-initialization instructions within a single transaction which mitigates this attack vector, future or third-party clients may not follow this pattern. Any client that splits mint initialization across multiple transactions inadvertently exposes applications to this vulnerability. Therefore, the program itself should enforce correct initialization semantics rather than relying on client behavior.

Code:

Repository: solana-program / token-2022

```
/// The ConfidentialTransferMint extension can be maliciously overwritten when its
initialization
/// instructions are executed across multiple transactions.
#[tokio::test]
async fn test_overwrite_ct_initialize_mint() {
    // Create keypairs
```

```
    let mint_account = Keypair::new();
    let mint_authority = Keypair::new();
    let decimals: u8 = 2;

    // Create a program test environment
    let mut program_test = ProgramTest::new("spl_token_2022", id(), None);
    program_test.add_program("spl_record", spl_record::id(), None);
    program_test.add_program("spl_elgamal_registry", spl_elgamal_registry::id(), N
one);
    let context = program_test.start_with_context().await;
    let context = Arc::new(Mutex::new(context));

    // Copy paste from init_token_with_mint_keypair_and_freeze_authority in progra
m_test.rs
    let payer = keypair_clone(&context.lock().await.payer);
    let client: Arc<dyn ProgramClient<ProgramBanksClientProcessTransaction>> =
        Arc::new(ProgramBanksClient::new_from_context(
            Arc::clone(&context),
            ProgramBanksClientProcessTransaction,
        ));

    let token = Token::new(Arc::clone(&client), &id(), &mint_account.pubkey(),
        Some(decimals), Arc::new(keypair_clone(&payer)),
    ).with_compute_unit_limit(ComputeUnitLimit::Simulated);

    // Create an empty mint account instruction
    let space = ExtensionType::try_calculate_account_len::<Mint>(&[
        ExtensionType::ConfidentialTransferMint]).unwrap();
    let create_account_ix = system_instruction::create_account(
        &payer.pubkey(),
        &mint_account.pubkey(),
        client.get_minimum_balance_for_rent_exemption(space)
            .await.map_err(TokenError::Client).unwrap(),
        space as u64,
        &id(),
    );

    // Create a Confidential Transfer InitializeMint instruction
    let ct_authority = Keypair::new();
    let ct_auditor_key = ElGamalKeypair::new_rand();
    let ct_init_mint_ix = ExtensionInitializationParams::ConfidentialTransferMint
{
        authority: Some(ct_authority.pubkey()),
        auto_approve_new_accounts: true,
        auditor_elgamal_pubkey: Some((*ct_auditor_key.pubkey()).into()),
    }
    .instruction(&id(), &mint_account.pubkey()).unwrap();
```

```
    // Process the create_account_ix and ct_init_mint_ix instructions into a trans
action
    token
        .process_ixs(&vec![create_account_ix, ct_init_mint_ix], &[&mint_account])
        .await.unwrap();

    // The attacker overwrites the Confidential Transfer mint values by creating a
new Confidential
    // Transfer InitializeMint instruction in a separate transaction.
    // No signature is required to perform this action.
    let no_signers: [&Keypair; 0] = [];
    let attacker_ct_authority = Keypair::new();
    let attacker_ct_auditor_key = ElGamalKeypair::new_rand();
    let attacker_ct_init_mint_ix = ExtensionInitializationParams::ConfidentialTran
sferMint {
        authority: Some(attacker_ct_authority.pubkey()),
        auto_approve_new_accounts: true,
        auditor_elgamal_pubkey: Some((*attacker_ct_auditor_key.pubkey()).into()),
    }
    .instruction(&id(), &mint_account.pubkey()).unwrap();
    token.process_ixs(&vec![attacker_ct_init_mint_ix], &no_signers)
        .await.unwrap();

    // Create a base InitializeMint instruction
    let init_mint_ix = instruction::initialize_mint(
        &id(), &mint_account.pubkey(), &mint_authority.pubkey(), None, decimals,
    ).unwrap();

    // Process the init_mint_ix instruction into a transaction
    token.process_ixs(&vec![init_mint_ix], &no_signers).await.unwrap();

    // Check the mint account to verify that the ConfidentialTransferMint extensio
n was overwritten
    let mint_state = token.get_mint_info().await.unwrap();
    let ct_mint = mint_state.get_extension::<ConfidentialTransferMint>()
        .unwrap();
    assert_eq!(ct_mint.authority.0, attacker_ct_authority.pubkey());
    assert!(ct_mint.auditor_elgamal_pubkey
        .equals(&Into::<PodElGamalPubkey>::into(*attacker_ct_auditor_key.pubkey()
)));
}
```

Recommendations:

Both of the following recommendations should be implemented together to fully mitigate the vulnerability and ensure safe initialization semantics.

1. Enforce Single Initialization - Prevent re-initialization of the `ConfidentialTransferMint` extension:

```
/// Processes an [`InitializeMint`] instruction.
fn process_initialize_mint(
    accounts: &[AccountInfo],
    authority: &OptionalNonZeroPubkey,
    auto_approve_new_account: PodBool,
    auditor_encryption_pubkey: &OptionalNonZeroElGamalPubkey,
) -> ProgramResult {
    let account_info_iter = &mut accounts.iter();
    let mint_info = next_account_info(account_info_iter)?;

    check_program_account(mint_info.owner)?;
    let mint_data = &mut mint_info.data.borrow_mut();
    let mut mint = PodStateWithExtensionsMut::<PodMint>::unpack_uninitialized(mint
_data)?;
    // Modification: Initialize the Confidential Transfer mint extension without a
llowing overwrite (overwrite input = false)
    let confidential_transfer_mint = mint.init_extension::<ConfidentialTransferMin
t>(false)?;

    confidential_transfer_mint.authority = *authority;
    confidential_transfer_mint.auto_approve_new_accounts = auto_approve_new_accoun
t;
    confidential_transfer_mint.auditor_elgamal_pubkey = *auditor_encryption_pubke
y;

    Ok(())
}
```

2. Revise the Mint Creation Workflow

Current workflow:

a. Allocate mint with enough space for extensions.

b. Populate extension fields.

c. Populate base mint fields and transition to Initialized.

Recommended workflow:

a. Allocate the mint account and populate base mint fields while keeping the account in uninitialized state.

b. Populate extension fields **requiring signatures from the mint authority** stored in the mint account.

c. Finally transition the mint to the Initialized state, preventing any further modification.

Also affected:

A similar issue exists in the initialization flow of the Confidential Mint Burn extension and the Confidential Transfer Fee extension. Just like `confidential_transfer::initialize_mint`, the `initialize_mint` instruction for those confidential extensions does **not enforce any authority check** and does not prevent re-initialization. As a result, an attacker could also front-run the legitimate initialization and overwrite

- the `supply_elgamal_pubkey` field for the Confidential Mint Burn extension, and
- the `withdraw_withheld_authority_elgamal_pubkey` for the Confidential Transfer Fee extension.

Impact for the Confidential Transfer Fee extension:

This creates a privacy-breaking impact. The withheld fee amount generated by each confidential transfer is encrypted using the `withdraw_withheld_authority_elgamal_pubkey`. If an attacker sets this key to a public key they control, they can decrypt the withheld fee amounts of all confidential transfers. Since the fee is deterministically derived from the underlying transfer amount, an attacker can infer information about the actual private transfer amount, partially breaking the confidentiality guarantees of the protocol.

Impact for the Confidential Mint Burn extension:

The security impact is significantly lower than for the other confidential extensions. The attacker cannot issue any Confidential Mint Burn instruction without signing as the mint authority. Since they do not control the mint authority, they cannot meaningfully exploit the overwritten key. At any time, the legitimate mint authority may restore control by invoking the `RotateSupplyElGamal` instruction, which safely updates the `supply_elgamal_pubkey` to a trusted value.

Applying the same initialization guarantees to the Confidential Mint Burn extension avoids edge-case behaviors, simplifies threat modeling, and ensures uniform security properties across all confidential extensions.

Recommendations:

For all confidential extensions, enforcing single-initialization semantics and proper authority signature checks during initialization would prevent this overwrite condition and strengthen confidentiality.

Issue: `ApplyPendingBalance` on frozen token account [#b03]

Severity: Medium

Description:

The `ApplyPendingBalance` instruction of the Token-2022 Confidential Transfer Account extension does not verify the freeze status of the token account. As a result, confidential-transfer balances may still be updated even though the account is not supposed to allow any state changes.

According to the Token-2022 specification, once a token account has been frozen, no instruction that alters its state should be executable. A frozen account must remain immutable until explicitly thawed. Allowing `ApplyPendingBalance` to bypass this rule constitutes a violation of the expected behavior defined in the spec.

Furthermore, this flaw causes **external explorers** and other transaction inspectors to incorrectly **interpret frozen accounts as active**, since confidential-transfer state transitions continue to occur despite the frozen

status. This misrepresentation can mislead auditors, monitoring tools, and downstream applications relying on on-chain state consistency.

Code:

Repository: solana-program / token-2022

```
mod program_test;
use {
    program_test::{
        ConfidentialTokenAccountBalances, ConfidentialTokenAccountMeta, TestContex
t, TokenContext,
    },
    solana_program_test::tokio,
    solana_sdk::signer::{keypair::Keypair, Signer},
    spl_token_2022_interface::extension::confidential_transfer::ConfidentialTransf
erMint,
    spl_token_2022_interface::extension::BaseStateWithExtensions,
    spl_token_client::token::ExtensionInitializationParams,
};

/// `ApplyPendingBalance` instruction successfully executes on a frozen token acco
unt.
#[tokio::test]
async fn test_apply_pending_balance_on_frozen_account() {
    // Create mint account with ConfidentialTransfer extension and freeze authorit
y
    let authority = Keypair::new();
    let freeze_authority = Keypair::new();

    let mut context = TestContext::new().await;
    context
        .init_token_with_mint_and_freeze_authority(
            vec![ExtensionInitializationParams::ConfidentialTransferMint {
                authority: Some(authority.pubkey()),
                auto_approve_new_accounts: true,
                auditor_elgamal_pubkey: None,
            }],
            Some(freeze_authority),
        ).await.unwrap();

    let TokenContext {
        freeze_authority,
        token,
        alice,
        decimals,
        mint_authority,
        ..
```

```
    } = context.token_context.unwrap();
    let freeze_authority = freeze_authority.unwrap();

    let mint_state = token.get_mint_info().await.unwrap();
    assert!(mint_state.get_extension::<ConfidentialTransferMint>().is_ok());
    let alice_meta =
        ConfidentialTokenAccountMeta::new(&token, &alice, Some(10), false, false).
await;

    let mint_amount = 1000;
    token.mint_to(&alice_meta.token_account, &mint_authority.pubkey(),
        mint_amount, &[&mint_authority]).await.unwrap();

    let deposit_amount = 500;
    token.confidential_transfer_deposit(&alice_meta.token_account,
        &alice.pubkey(), deposit_amount, decimals, &[&alice]).await.unwrap();

    alice_meta.check_balances(
            &token,
            ConfidentialTokenAccountBalances {
                pending_balance_lo: deposit_amount,
                pending_balance_hi: 0,
                available_balance: 0,
                decryptable_available_balance: 0,
            },
        ).await;

    // Frozen Alice's account
    token.freeze(&alice_meta.token_account, &freeze_authority.pubkey(),
        &[&freeze_authority]).await.unwrap();

    // Apply pending balance to Alice's account in order to move tokens from pendi
ng to available balance
    // This instruction should fail because Alice's account is frozen, but current
ly it succeeds.
    token.confidential_transfer_apply_pending_balance(
            &alice_meta.token_account, &alice.pubkey(), None,
            alice_meta.elgamal_keypair.secret(), &alice_meta.aes_key, &[&alice],
        ).await.unwrap();

    alice_meta.check_balances(
            &token,
            ConfidentialTokenAccountBalances {
                pending_balance_lo: 0,
                pending_balance_hi: 0,
                available_balance: deposit_amount,
                decryptable_available_balance: deposit_amount,
```

```
            },
        ).await;
}
```

Recommendations:

Add a check in `process_apply_pending_balance` to ensure the token account is not frozen before applying any pending balance updates.

```
/// Processes an [`ApplyPendingBalance`] instruction.
#[cfg(feature = "zk-ops")]
fn process_apply_pending_balance(
    program_id: &Pubkey,
    accounts: &[AccountInfo],
    ApplyPendingBalanceData {
        expected_pending_balance_credit_counter,
        new_decryptable_available_balance,
    }: &ApplyPendingBalanceData,
) -> ProgramResult {
    let account_info_iter = &mut accounts.iter();
    let token_account_info = next_account_info(account_info_iter)?;
    let authority_info = next_account_info(account_info_iter)?;
    let authority_info_data_len = authority_info.data_len();

    check_program_account(token_account_info.owner)?;
    let token_account_data = &mut token_account_info.data.borrow_mut();
    let mut token_account = PodStateWithExtensionsMut::<PodAccount>::unpack(token_
account_data)?;

    // Check that the token account is not frozen
    if token_account.base.is_frozen() {
         return Err(TokenError::AccountFrozen.into());
     }
     ...
}
```

Also affected:

The same issue applies to the `ApproveAccount` instruction, which can also be executed successfully on a frozen token account, even though it modifies account state. This should similarly be prevented to conform with the specification governing frozen accounts.

Issue: Incoherent Extension Combination Allowed: `NonTransferable` and `ConfidentialTransfer` without `ConfidentialMintBurn` [#b04]

Severity: Medium

Description:

The SPL Token program allows enabling `NonTransferable` and `ConfidentialTransferMint` together without requiring `ConfidentialMintBurn`, creating an incoherent configuration. The mint appears to support confidential functionality, but deposits into confidential balances always fail due to the non-transferable restriction, and confidential balances cannot be properly managed without the mint-burn extension. This leads to misleading token configurations that can never function as expected. This reflects a design flaw that can mislead users about what the token is actually capable of.

Code:

Repository: solana-program / token-2022

```
#[tokio::test]
async fn test_non_transferable_and_confidential_transfer() {
    // Create mint account with NonTransferable and Confidential Transfer extensio
ns
    let authority = Keypair::new();
    let mut context = TestContext::new().await;
    context
        .init_token_with_mint(vec![
            ExtensionInitializationParams::ConfidentialTransferMint {
                authority: Some(authority.pubkey()),
                auto_approve_new_accounts: true,
                auditor_elgamal_pubkey: None,
            },
            ExtensionInitializationParams::NonTransferable,
        ]).await.unwrap();

    let TokenContext {
        token,
        alice,
        decimals,
        mint_authority,
        ..
    } = context.token_context.unwrap();

    {
        // Check that NonTransferable and ConfidentialTransfer are enabled.
        let mint_state = token.get_mint_info().await.unwrap();
        assert!(mint_state.get_extension::<NonTransferable>().is_ok());
        assert!(mint_state
            .get_extension::<ConfidentialTransferMint>()
            .is_ok());
    }

    let alice_meta =
        ConfidentialTokenAccountMeta::new(&token, &alice, Some(10), false, false).
await;
```

```
    let mint_amount = 1000;
    token
        .mint_to(
            &alice_meta.token_account,
            &mint_authority.pubkey(),
            mint_amount,
            &[&mint_authority],
        ).await.unwrap();

    {
        // Check that transferring tokens into a confidential balance is disallowe
d
        let deposit_amount = 500;
        let deposit_err = token
            .confidential_transfer_deposit(
                &alice_meta.token_account,
                &alice.pubkey(),
                deposit_amount,
                decimals,
                &[&alice],
            ).await.unwrap_err();
        assert_eq!(
            deposit_err,
            TokenClientError::Client(Box::new(TransportError::TransactionError(
                TransactionError::InstructionError(
                    0,
                    InstructionError::Custom(TokenError::NonTransferable as u32)
                )
            )))
        );
    }
}
```

Recommendations:

The program should enforce coherent extension combinations for non-transferable mints.
Only the following two configurations should be permitted:

- Enable `NonTransferable` only for public non-transferable tokens
- Enable `NonTransferable`, `ConfidentialTransferMint`, and `ConfidentialMintBurn` for private non-transferable tokens

The combination `NonTransferable` and `ConfidentialTransferMint` without `ConfidentialMintBurn` should be rejected at initialization.

Issue: Client/Interface Allow Multisig CT Authority While Program Rejects It [#b05]

Severity: Medium

Description:

The SPL Token 2022 program does not support using a multisig account as the Confidential Transfer mint authority for the `UpdateMint` instruction (code), even though both the client (code) and the interface (code) suggest that multisig authorities are allowed.

This mismatch introduces no security risk (the program safely rejects the transaction) but it creates a design inconsistency that can confuse users. This creates the **false impression that CT mint configuration can be managed via multisig.**

Also affected:

The same multisig mismatch occurs for the `ApproveAccount` instruction, which likewise does not support multisig authorities even though the client and interface appear to allow it.

Code:

Repository: solana-program / token-2022

```
/// Multi-signers is not supported for UpdateMint instruction.
#[tokio::test]
async fn test_update_mint_with_multisig() {
    let mut context = TestContext::new().await;

    // Create multisig account
    let ctx = context.context.lock().await;
    let payer = ctx.payer.insecure_clone();
    let blockhash = ctx.last_blockhash;
    let rent = ctx.banks_client.get_rent().await.unwrap();

    let signer1 = Keypair::new();
    let signer2 = Keypair::new();
    let signer3 = Keypair::new();
    let minimum_signers = 2;
    let multisig = Keypair::new();
    let multisig_space = <Multisig as solana_program_pack::Pack>::LEN;
    let multisig_lamports = rent.minimum_balance(multisig_space);

    let create_multisig_ix = system_instruction::create_account(
        &payer.pubkey(),
        &multisig.pubkey(),
        multisig_lamports,
        multisig_space as u64,
        &spl_token_2022_interface::ID,
    );
    let init_multisig_ix = token_instruction::initialize_multisig(
        &spl_token_2022::id(),
        &multisig.pubkey(),
```

```
            &[&signer1.pubkey(), &signer2.pubkey(), &signer3.pubkey()],
            minimum_signers,
        ).unwrap();
        let tx = Transaction::new_signed_with_payer(
            &[create_multisig_ix, init_multisig_ix],
            Some(&payer.pubkey()),
            &[&payer, &multisig],
            blockhash,
        );
        ctx.banks_client.process_transaction(tx).await.unwrap();
        drop(ctx);

        // Create mint account with CT extension
        context
            .init_token_with_mint(vec![
                ExtensionInitializationParams::ConfidentialTransferMint {
                    authority: Some(multisig.pubkey()),
                    auto_approve_new_accounts: true,
                    auditor_elgamal_pubkey: None,
                },
            ]).await.unwrap();

        let TokenContext { token, .. } = context.token_context.unwrap();

        {
            // Cannot sign the UpdateMint instruction with a multisig CT authority
            let update_mint_err = token
                .confidential_transfer_update_mint(
                    &multisig.pubkey(),
                    false,
                    None,
                    &[&signer1, &signer2, &signer3],
                ).await.unwrap_err();
            assert_eq!(
                update_mint_err,
                TokenClientError::Client(Box::new(TransportError::TransactionError(
                    TransactionError::InstructionError(0, InstructionError::MissingReq
uiredSignature)
                )))
            );
        }
    }
```

Recommendations:

The `UpdateMint` and `ApproveAccount` instructions in the confidential transfer extension processor should use the standard `validate_owner` function (which properly handles both single-signer and multisig authorities) instead

of manually checking `is_signer`. Both the client library and instruction interface already support multisig for these operations.

Issue: Redundant Serialization in Compound Ciphertext Operations [#b06]

Severity: Low impact / best practices (code quality, performance)

Description:

The `add_with_lo_hi()` and `subtract_with_lo_hi()` functions chain primitive operations that each perform complete deserialize-compute-serialize cycles. Intermediate results are serialized then immediately deserialized by the next operation, creating redundant overhead. Each call performs 5 deserializations and 3 serializations when only 3 and 1 are required respectively—representing significant serialization and deserialization overhead.

These functions are invoked on every confidential balance update: `subtract_with_lo_hi` debits the sender, `add_with_lo_hi` credits the receiver's pending balance, and `add_with_lo_hi` is called again when applying pending balances. A single end-to-end confidential transfer invokes these operations 3-4 times, compounding the overhead across every confidential token movement in the system.

Code:

Repository: zk-elgamal-proof

File: `token-2022/confidential/ciphertext-arithmetic/src/lib.rs`

```
pub fn add_with_lo_hi(
    left_ciphertext: &PodElGamalCiphertext,
    right_ciphertext_lo: &PodElGamalCiphertext,
    right_ciphertext_hi: &PodElGamalCiphertext,
) -> Option<PodElGamalCiphertext> {
    let shift_scalar = u64_to_scalar(1_u64 << SHIFT_BITS);
    let shifted_right_ciphertext_hi = multiply(&shift_scalar, right_ciphertext_h
i)?;
    //   deser(hi) + compute + ser(shifted)

    let combined_right_ciphertext = add(right_ciphertext_lo, &shifted_right_cipher
text_hi)?;
    //   deser(lo) + deser(shifted) + compute + ser(combined)

    add(left_ciphertext, &combined_right_ciphertext)
    //   deser(left) + deser(combined) + compute + ser(result)
```

and

```
pub fn subtract_with_lo_hi(
    left_ciphertext: &PodElGamalCiphertext,
    right_ciphertext_lo: &PodElGamalCiphertext,
```

```
    right_ciphertext_hi: &PodElGamalCiphertext,
) -> Option<PodElGamalCiphertext> {
    let shift_scalar = u64_to_scalar(1_u64 << SHIFT_BITS);
    let shifted_right_ciphertext_hi = multiply(&shift_scalar, right_ciphertext_h
i)?;
    //  deser(hi) + compute + ser(shifted)

    let combined_right_ciphertext = add(right_ciphertext_lo, &shifted_right_cipher
text_hi)?;
    //   deser(lo) + deser(shifted) + compute + ser(combined)

    subtract(left_ciphertext, &combined_right_ciphertext)
    //   deser(left) + deser(combined) + compute + ser(result)
}
```

Recommendations:

Introduce internal helpers operating on Ristretto point tuples. Deserialize all three input ciphertexts once at entry, perform the scalar multiplication and additions directly, and serialize only the final result. Additionally, consider eliminating the base64 encode/decode workaround in `ristretto_to_elgamal_ciphertext`, which converts bytes to a base64 string and parses it back via `FromStr`. A direct byte constructor for `PodElGamalCiphertext` would remove this unnecessary round-trip.

Issue: Resource Exhaustion via Proof Verification Order [#b07]

Severity: Low impact / best practices

Description:

Functions performing confidential transfer operations execute expensive proof verification before cheap authorization checks (owner validation, signer verification). An attacker can submit valid proofs targeting accounts they don't own, wasting compute resources before the authorization check rejects the transaction.

Partially mitigated by the Solana fee mechanism.

Code:

Repository: token-2022

File: confidential/elgamal-registry/src/processor.rs

```
  fn process_update_registry_account(...) -> ProgramResult {
      let elgamal_registry_account_info = next_account_info(account_info_iter)?;
      let elgamal_registry_account = pod_from_bytes_mut::<ElGamalRegistry>(...)?;

      // expensive runs first
      let proof_context = verify_and_extract_context::<...>(...)?;

      // cheap runs after
      let owner_info = next_account_info(account_info_iter)?;
```

```
        validate_registry_owner(owner_info, &elgamal_registry_account.owner)?;
    }
```

Also Affected:

- confidential/elgamal-registry/src/processor.rs - `process_create_registry_account`
- program/src/extension/confidential_transfer/processor.rs - `process_configure_account`
- program/src/extension/confidential_transfer/processor.rs - `process_empty_account`
- program/src/extension/confidential_transfer/processor.rs - `process_withdraw`
- program/src/extension/confidential_transfer/processor.rs - `process_transfer`
- program/src/extension/confidential_transfer_fee/processor.rs - `process_withdraw_withheld_tokens_from_mint`
- program/src/extension/confidential_transfer_fee/processor.rs - `process_withdraw_withheld_tokens_from_accounts`
- program/src/extension/confidential_mint_burn/processor.rs - `process_rotate_supply_elgamal_pubkey`
- program/src/extension/confidential_mint_burn/processor.rs - `process_confidential_mint`
- program/src/extension/confidential_mint_burn/processor.rs - `process_confidential_burn`

Recommendations:

Perform cheap checks first in all cases:

file: confidential/elgamal-registry/src/processor.rs

```
fn process_update_registry_account(...) -> ProgramResult {
    let elgamal_registry_account_info = accounts.first().ok_or(ProgramError::NotEn
oughAccountKeys)?;
    let owner_info = accounts.get(2).ok_or(ProgramError::NotEnoughAccountKeys)?;

    // validate first
    validate_program_owner(elgamal_registry_account_info, program_id)?;
    let elgamal_registry_account = pod_from_bytes_mut::<ElGamalRegistry>(...)?;
    validate_registry_owner(owner_info, &elgamal_registry_account.owner)?;

    // expensive checks only after authorization passes
    let proof_context = verify_and_extract_context::<...>(...)?;
}
```

The Confidential Transfer Fee extension

Issue: Incorrect Pending Balance Check in Withheld Token Withdrawals [#b08]

Severity: Medium

Description:

The `WithdrawWithheldTokensFromMint` instruction credits tokens directly to the available balance of the destination account and does not update the pending balance or its associated credit counter. Therefore, the instruction should not fail when the destination account's `pending_balance_credit_counter` equals its `maximum_pending_balance_credit_counter`, since no pending credit is created.

Applying this check incorrectly leads to unnecessary transaction failures and can prevent the `TransferFeeConfig` withdraw-withheld authority from distributing withheld fees to certain accounts. Moreover, because this authority cannot invoke `ApplyPendingBalance` on accounts it does not own, it has no way to reduce their `pending_balance_credit_counter`, meaning such accounts may be permanently unable to receive withheld-fee withdrawals even though the operation is semantically valid.

The workaround in this situation is to call `ApplyPendingBalance` to reset the counter, then the withdraw succeeds. However, **the user probably will not be aware of this option** and the error message does not suggest any help.

Code:

Repository: solana-program / token-2022

```
const TEST_MAXIMUM_FEE: u64 = 100;
const TEST_FEE_BASIS_POINTS: u16 = 250;

#[tokio::test]
async fn test_withdraw_withheld_with_max_pending_counter() {
    let withdraw_withheld_authority = Keypair::new();
    let auditor_elgamal_keypair = ElGamalKeypair::new_rand();
    let withdraw_withheld_authority_elgamal_keypair = ElGamalKeypair::new_rand();

    let mut context = TestContext::new().await;
    context
        .init_token_with_mint(vec![
            ExtensionInitializationParams::TransferFeeConfig {
                transfer_fee_config_authority: Some(Keypair::new().pubkey()),
                withdraw_withheld_authority: Some(withdraw_withheld_authority.pubk
ey()),
                transfer_fee_basis_points: TEST_FEE_BASIS_POINTS,
                maximum_fee: TEST_MAXIMUM_FEE,
            },
            ExtensionInitializationParams::ConfidentialTransferMint {
                authority: Some(Keypair::new().pubkey()),
                auto_approve_new_accounts: true,
                auditor_elgamal_pubkey: Some((*auditor_elgamal_keypair.pubkey()).i
nto()),
            },
            ExtensionInitializationParams::ConfidentialTransferFeeConfig {
                authority: Some(Keypair::new().pubkey()),
                withdraw_withheld_authority_elgamal_pubkey: (*withdraw_withheld_au
thority_elgamal_keypair.pubkey()).into(),
            },
```

```
        ]).await.unwrap();

    let TokenContext { token, alice, bob, decimals, mint_authority, .. } = contex
t.token_context.unwrap();

    // Create accounts with max_pending_balance_credit_counter = 1 for demonstrati
on purposes.
    // The bug is still valid with the default limit
    let alice_meta = ConfidentialTokenAccountMeta::new(&token, &alice, Some(1), fa
lse, true).await;
    let bob_meta = ConfidentialTokenAccountMeta::new(&token, &bob, Some(1), false,
true).await;

    // Fund Alice and move to available balance
    token.mint_to(&alice_meta.token_account, &mint_authority.pubkey(), 1000, &[&mi
nt_authority]).await.unwrap();
    token.confidential_transfer_deposit(&alice_meta.token_account, &alice.pubkey
(), 500, decimals, &[&alice]).await.unwrap();
    token.confidential_transfer_apply_pending_balance(&alice_meta.token_account, &
alice.pubkey(), None, alice_meta.elgamal_keypair.secret(), &alice_meta.aes_key, &
[&alice]).await.unwrap();

    // Transfer to Bob
    token.confidential_transfer_transfer_with_fee(
        &alice_meta.token_account, &bob_meta.token_account, &alice.pubkey(),
        None, None, None, None, None, 100, None,
        &alice_meta.elgamal_keypair, &alice_meta.aes_key, &bob_meta.elgamal_keypai
r.pubkey(),
        Some(auditor_elgamal_keypair.pubkey()), withdraw_withheld_authority_elgama
l_keypair.pubkey(),
        TEST_FEE_BASIS_POINTS.into(), TEST_MAXIMUM_FEE.into(), &[&alice],
    ).await.unwrap();

    // Max out Alice pending counter with another deposit
    token.confidential_transfer_deposit(&alice_meta.token_account, &alice.pubkey
(), 100, decimals, &[&alice]).await.unwrap();

    // Verify pending counter is at max
    let state = token.get_account_info(&alice_meta.token_account).await.unwrap();
    let ct_state = state.get_extension::<ConfidentialTransferAccount>().unwrap();
    assert_eq!(ct_state.maximum_pending_balance_credit_counter, ct_state.pending_b
alance_credit_counter);

    // WithdrawWithheldTokensFromAccounts fails with MaximumPendingBalanceCreditCo
unterExceeded
    let err = token.confidential_transfer_withdraw_withheld_tokens_from_accounts(
        &alice_meta.token_account, &withdraw_withheld_authority.pubkey(), None, No
```

```
ne,
        &withdraw_withheld_authority_elgamal_keypair, &alice_meta.elgamal_keypair.
pubkey(),
        &DecryptableBalance::default(), &[&bob_meta.token_account], &[&withdraw_wi
thheld_authority],
    ).await.unwrap_err();

    assert_eq!(
        err,
        TokenClientError::Client(Box::new(TransportError::TransactionError(
            TransactionError::InstructionError(0, InstructionError::Custom(TokenEr
ror::MaximumPendingBalanceCreditCounterExceeded as u32))
        )))
    );
}
```

Recommendations:

- Create a new validation method that only checks approval and `allow_confidential_credits`,

OR

- Remove only the counter check from both instructions while keeping the other validations

Also affected:

`WithdrawWithheldTokensFromAccounts` is subject to the same issue.
This instruction also credits withheld amounts directly to the destination account's available balance without modifying pending balance or credit counters. Therefore, it should not enforce any checks related to pending-balance saturation. Applying such checks could similarly block legitimate withheld-fee distributions and lead to permanent inability to credit certain accounts.

The Confidential Mint and Burn extension

Issue: Close a mint account with a non zero confidential supply [#b09]

Severity: Critical

Description:

When the `ConfidentialMintBurn` extension is enabled, the mint account maintains both a public `supply` and a `confidential_supply` encrypted under ElGamal. However, the SPL Token program does not verify that the `confidential_supply` equals zero before allowing the mint account to be closed.

As a result, a mint with a non-zero confidential supply can be prematurely closed, removing the only on-chain record needed to authorize confidential operations. Once the mint account is closed, users can no longer perform confidential transfer or burn operations. This results in an inconsistent state where confidential tokens may still exist on-chain but can no longer be utilized.

The core issue arises from an inconsistency in the design between public and confidential tokens. For standard SPL tokens without the confidential extension, the mint account cannot be closed unless the total

supply is zero, which provides clear guarantees to users about the conditions under which the close authority may act. In contrast, when the `ConfidentialMintBurn` extension is enabled, the SPL Token program does not enforce that the confidential supply be zero prior to closing the mint. This disparity can **mislead users and mint authorities into assuming that the same safety property holds in both cases, leaving them unaware that the close authority can legitimately close a mint even while confidential tokens still effectively exist**. As a result, users may form incorrect expectations about the lifecycle and safety conditions of confidential token mints. Also, the inconsistent design may result with unintentional loss of fund.

Code:

Repository: solana-program / token-2022

File: `clients/rust-legacy/tests/confidential_mint_burn.rs`

```
/// `CloseAccount` instruction successfully executes on a mint token account with
a non-zero
/// confidential supply.
#[tokio::test]
async fn test_close_mint_account_with_non_zero_confidential_supply() {
    use spl_token_2022_interface::instruction;

    let close_authority = Keypair::new();
    let confidential_transfer_authority = Keypair::new();
    let auto_approve_new_accounts = true;
    let auditor_elgamal_keypair = ElGamalKeypair::new_rand();
    let auditor_elgamal_pubkey = (*auditor_elgamal_keypair.pubkey()).into();
    let supply_elgamal_keypair = ElGamalKeypair::new_rand();
    let supply_elgamal_pubkey = (*supply_elgamal_keypair.pubkey()).into();
    let supply_aes_key = AeKey::new_rand();
    let decryptable_supply = supply_aes_key.encrypt(0).into();

    let mut context = TestContext::new().await;
    context
        .init_token_with_mint(vec![
            ExtensionInitializationParams::ConfidentialTransferMint {
                authority: Some(confidential_transfer_authority.pubkey()),
                auto_approve_new_accounts,
                auditor_elgamal_pubkey: Some(auditor_elgamal_pubkey),
            },
            ExtensionInitializationParams::ConfidentialMintBurn {
                supply_elgamal_pubkey,
                decryptable_supply,
            },
            ExtensionInitializationParams::MintCloseAuthority {
                close_authority: Some(close_authority.pubkey()),
            },
        ]).await.unwrap();

    let TokenContext {
```

```
        token,
        mint_authority,
        alice,
        bob,
        ..
    } = context.token_context.unwrap();

    let alice_meta = ConfidentialTokenAccountMeta::new(&token, &alice).await;
    let bob_meta = ConfidentialTokenAccountMeta::new(&token, &bob).await;

    let mint_amount = 120;
    mint_with_option(
        &token,
        &mint_authority.pubkey(),
        &alice_meta.token_account,
        mint_amount,
        &supply_elgamal_keypair,
        alice_meta.elgamal_keypair.pubkey(),
        Some(auditor_elgamal_keypair.pubkey()),
        &supply_aes_key,
        &[&mint_authority],
        ConfidentialTransferOption::InstructionData,
    ).await.unwrap();

    token
        .confidential_transfer_apply_pending_balance(
            &alice_meta.token_account,
            &alice.pubkey(),
            None,
            alice_meta.elgamal_keypair.secret(),
            &alice_meta.aes_key,
            &[&alice],
        ).await.unwrap();

    {
        // Check that the confidential supply is equal to mint_amount
        let state = token.get_mint_info().await.unwrap();
        let extension = state.get_extension::<ConfidentialMintBurn>().unwrap();
        let confidential_supply: ElGamalCiphertext =
            extension.confidential_supply.try_into().unwrap();
        assert_eq!(
            confidential_supply.decrypt_u32(supply_elgamal_keypair.secret())
                .unwrap(),
            mint_amount
        );
    }
```

```
    // Close the mint account
    // This instruction should fail because the confidential supply of the mint
    // account is not equal to zero, but currently it succeeds.
    let payer = Keypair::new_from_array(*context.context.lock().await.payer.secret
_bytes());
    let destination = Pubkey::new_unique();
    token
        .process_ixs(
            &[instruction::close_account(
                &spl_token_2022_interface::id(),
                token.get_address(),
                &destination,
                &close_authority.pubkey(),
                &[],
            ).unwrap()],
            &[&close_authority, &payer],
        ).await.unwrap();

    {
        // Check that the mint account is closed
        let mint_err = token.get_mint_info().await.unwrap_err();
        assert_eq!(mint_err, TokenClientError::AccountNotFound);
    }

    {
        // Check that Alice cannot send tokens to Bob
        let transfer_amount = 50;
        let transfer_err = token
            .confidential_transfer_transfer(
                &alice_meta.token_account,
                &bob_meta.token_account,
                &alice.pubkey(),
                None,
                None,
                None,
                transfer_amount,
                None,
                &alice_meta.elgamal_keypair,
                &alice_meta.aes_key,
                &bob_meta.elgamal_keypair.pubkey(),
                Some(auditor_elgamal_keypair.pubkey()),
                &[&alice],
            ).await.unwrap_err();
        assert_eq!(transfer_err, TokenClientError::AccountNotFound);
    }

    {
```

```
        // Check that Alice cannot burn tokens
        let burn_amount = 50;
        let burn_err = token
            .confidential_transfer_burn(
                &alice.pubkey(),
                &alice_meta.token_account,
                None,
                None,
                None,
                burn_amount,
                &alice_meta.elgamal_keypair,
                supply_elgamal_keypair.pubkey(),
                Some(auditor_elgamal_keypair.pubkey()),
                &alice_meta.aes_key,
                None,
                &[&alice],
            ).await.unwrap_err();
        assert_eq!(
            burn_err,
            TokenClientError::Client(Box::new(TransportError::TransactionError(
                TransactionError::InstructionError(0, InstructionError::IncorrectP
rogramId)
            )))
        );
    }
}
```

Recommendations:

The program should prevent closing a mint when the Confidential Mint Burn extension reports a non-zero `confidential_supply`. Requiring the confidential supply to be zero before mint closure ensures consistent supply-safety semantics and prevents leaving unburned confidential tokens in circulation.

```
impl ConfidentialMintBurn {
    /// Checks if the mint can be closed based on confidential supply state
   pub fn closable(&self) -> ProgramResult {
        // No pending_burn check needed: when confidential_supply is zero,
        // no confidential tokens remain in circulation, therefore pending_burn
        // cannot be non-zero.
        if self.confidential_supply == EncryptedBalance::zeroed()
        {
            Ok(())
        } else {
            Err(TokenError::MintHasSupply.into())
        }
    }
}
```

```
pub fn process_close_account(program_id: &Pubkey, accounts: &[AccountInfo]) -> Pro
gramResult {
        ...

        let source_account_data = source_account_info.data.borrow();
        if let Ok(source_account) =
            PodStateWithExtensions::<PodAccount>::unpack(&source_account_data)
        {
            ...
        } else if let Ok(mint) = PodStateWithExtensions::<PodMint>::unpack(&source
_account_data) {
            let extension = mint.get_extension::<MintCloseAuthority>()?;
            let maybe_authority: Option<Pubkey> = extension.close_authority.into
();
            let authority = maybe_authority.ok_or(TokenError::AuthorityTypeNotSupp
orted)?;
            Self::validate_owner(
                program_id,
                &authority,
                authority_info,
                authority_info_data_len,
                account_info_iter.as_slice(),
            )?;

            if u64::from(mint.base.supply) != 0 {
                return Err(TokenError::MintHasSupply.into());
            }

            if let Ok(confidential_mint_burn) =
                source_account.get_extension::<ConfidentialMintBurn>()
            {
                confidential_mint_burn.closable()?;
            }
        } else {
            return Err(ProgramError::UninitializedAccount);
        }
       ...
        Ok(())
    }
```

The the zk-elgamal-proof program

continued from the intermediately report

Issue: Missing Chain-Specific Personalization [#b10]

As discussed with the Anza team

Severity: Medium (High in terms of user safety)

Description:

All proof transcripts in this repository use hardcoded domain strings (e.g., `b"ciphertext-ciphertext-equality-instruction"`, `b"zero-ciphertext-instruction"`, etc.) without a shared chain-specific personalization prefix.
This affects every proof type: equality proofs, validity proofs, range proofs, and zero-ciphertext proofs. Proofs generated on one blockchain can be replayed on any fork or chain using this code, as transcript domains are identical across all deployments.

Code:

Repository: zk-elgamal-proof

Recommendations:

Define a single global personalization constant in `src/lib.rs` that ALL proof transcripts must use:

```
// src/lib.rs
/// Global transcript domain separator.
pub const TRANSCRIPT_DOMAIN: &[u8] = b"solana-zk-sdk-v1";
```

Update every `new_transcript()` implementation across all proof types to use this constant:

```
fn new_transcript(&self) -> Transcript {
    let mut transcript = Transcript::new(TRANSCRIPT_DOMAIN);
    transcript.append_message(b"ciphertext-ciphertext-equality-instruction", b"");
    // ... append context
}
```

Document this prominently in the repository's top-level `README` with a dedicated "**Security: Fork Requirements**" section explaining that forks MUST change `TRANSCRIPT_DOMAIN` to a unique value before deployment to prevent cross-chain proof replay attacks.

Issue: `BatchedRangeProofU256Data` stated to validate bit_length ≤ 128 but `RangeProof::new` enforces bit_length ≤ 64 [#b11]

Severity: Medium

Description:

This is a correctness issue proving a logical contradiction exists between the validation logic of the high-level `BatchedRangeProofU256Data::new` wrapper and the low-level `RangeProof::new` engine it calls. On one hand, the `u256` wrapper (in the `batched_range_proof_u256.rs` file) allows for individual bit_lengths up to 128. On the other

hand, it passes these inputs to the `RangeProof::new` engine (range-proof/mod.rs file), which is hard-coded to only accept `u64` amounts and explicitly rejects any bit_length greater than 64.

This makes the u256 wrapper functionally unusable for proving values with bit-lengths in the interval `(64, 128]`. This means that, for example, a developer attempting to prove a 96-bit value will have their inputs accepted by the wrapper, only for the underlying prover engine to fail, returning an `InvalidBitSize` error and wasting prover resources.

Code:

Repository: zk-elgamal-proof
File: `zk-elgamal-proof-program/proof-data/batched-range-proof/batched_range_proof_u256.rs`

```
#[cfg(test)]
mod test {
    use {
        ...
        crate::{
            range_proof::errors::RangeProofGenerationError,
        ...
        },

    };
#[test]
    fn bprpu256_prover_logic_contradiction() {
        //  For exemplification, a 96-bit lenght is used; note that any integer in
the interval (64, 128) produces the same outcome.
        //  A 96-bit lenght is valid for the U256 wrapper which allows up to 128 b
it lengths but is invalid for the
        //  inner `RangeProof` engine which allows max 64 bit lengths.
        let amount: u64 = 100;
        let (comm, open) = Pedersen::new(amount);

        // Add padding to satisfy the 256-bit sum
        let (comm_pad, open_pad) = Pedersen::new(0_u64);

        // Call the U256 wrapper
        let proof_result = BatchedRangeProofU256Data::new(
            vec![&comm, &comm_pad, &comm_pad, &comm_pad],
            vec![amount, 0, 0, 0],
            vec![96, 64, 64, 32], // Sum to 256
            vec![&open, &open_pad, &open_pad, &open_pad],
        );

        // Incorrect implementation confirmed: the prover fails.
        // The root cause: the error is `InvalidBitSize`, which
        // comes from the inner `RangeProof::new`, proving the logical contradicti
on.
        assert!(matches!(
```

```
            proof_result,
            Err(ProofGenerationError::RangeProof(RangeProofGenerationError::Invali
dBitSize))
        ));
    }
```

Recommendations:

There are two paths. Either:

1. Modify `batched_range_proof_u256.rs` to enforce the `u64::BITS` limit. This makes the code consistent but prevents the `u256` wrapper from proving components larger than 64 bits.

Or

2. Refactor both `BatchedRangeProofU256Data::new` and `RangeProof::new` to accept `u128` amounts, update the bit extraction logic accordingly, and remove the `u64::BITS` constraint. This would enable proving individual components up to 128 bits while maintaining the 256-bit total.

Issue: Fail to Detect Mutated Context on Range Proof Verification [#b12]

Severity: Medium

Description:

The `TryInto` implementation for `BatchedRangeProofContext` is non-canonical and it causes two different serialized contexts (a clean, canonical one and a mutated one with spurious data after the first zero sentinel) to successfully decode into the exact same set of commitments and bit-lengths.

The current proof verifier (`verify_proof`) succeeds in accepting a proof for a mutated context (i.e., a context that is non-canonical) when the transcript has been adjusted to account for that, as described in the test below.

Code:

Repository: zk-elgamal-proof

File: `zk-sdk/src/zk-elgamal-proof-program/proof-data/batched-range-proof/batched_range_proof_u64.rs`

```
#[cfg(test)]
mod test {
    use {
        super::*,
        crate::{
            ...,
            range_proof::{..., RangeProof},
            zk_elgamal_proof_program::{..., proof_data::batched_range_proof::PodPe
dersenCommitment},
        },...
```

```
     };
#[test]
fn proof_bound_to_mutated_context_verifies() {
    let amount = 42u64;
    let (c1, o1) = Pedersen::new(amount);

    // Start from canonical context, then make it non-canonical by adding spurious
data after sentinel.
    let canonical = BatchedRangeProofU64Data::new(
        vec![&c1],
        vec![amount],
        vec![64usize],
        vec![&o1],
    ).expect("construct proof");

    let mut mutated_ctx = canonical.context;
    mutated_ctx.bit_lengths[1] = 1;
    mutated_ctx.bit_lengths[2] = 63;
    mutated_ctx.commitments[2] = mutated_ctx.commitments[0];

    // Prover binds transcript to the mutated context and produces a range proof.
    let mut transcript = mutated_ctx.new_transcript();
    let proof = RangeProof::new(vec![amount], vec![64usize], vec![&o1], &mut trans
cript)
        .expect("proof generation")
        .try_into()
        .expect("to PodRangeProofU64");

    // Verifier uses the same mutated serialization. It must pass.
    let proof_data = BatchedRangeProofU64Data { context: mutated_ctx, proof };
    assert!(proof_data.verify_proof().is_ok());
}
```

Recommendations:

We recommend a stricter de-serialization logic: Modify the `TryInto` implementation for `BatchedRangeProofContext` to be canonical. In particular, after parsing the commitments with `take_while`, iterate over the remaining elements in both commitments and bit_lengths and assert that they are all zero.

Additionally, `TryInto` should also validate that the parsed bit_lengths do not exceed the allowed `u64::BITS (64)`, a check that is missing in the current `TryInto` implementation.

Issue: Invalid Sigma Proof Generation from Unchecked Parameters [#b13]

Severity: Medium

Description:

The core `CiphertextCiphertextEqualityProof::new` function and its high-level wrapper, `CiphertextCiphertextEqualityProofData::new`, fail to validate the cryptographic consistency of their inputs. Both functions accept a set of cryptographic components (`first_keypair`, `first_ciphertext`, `second_pubkey`, `second_ciphertext`, `second_opening`, `amount`) but do not verify that these components are consistent with each other. The API's contract implies that `first_ciphertext` and `second_ciphertext` both encrypt the *same* `amount` using the provided keys and randomness.

This contract is violated. The functions blindly trust the caller. A developer can pass mismatched data (e.g., `amount = 50`, but a `second_ciphertext` that encrypts `100`), and the function will incorrectly return `Ok(proof_data)`. The resulting proof is cryptographically invalid, will always fail on-chain verification, and causes wasted prover resources, transaction fees, and difficult-to-debug errors.

Analogous issues are present in all sigma_proofs and proof_data files implementing arguments of knowledge.

Code:

Repository: zk-elgamal-proof

File: `zk-sdk/src/sigma_proofs/ciphertext_ciphertext_equality.rs`

```
#[test]
   fn test_sigma_proof_accepts_mismatched_inputs() {
       // Setup: two keypairs and two distinct amounts
       let first_keypair = ElGamalKeypair::new_rand();
       let second_keypair = ElGamalKeypair::new_rand();

       let amount_1: u64 = 50;
       let amount_2: u64 = 100;

       // Create the ciphertexts corresponding to the two distinct amounts
       let first_ciphertext = first_keypair.pubkey().encrypt(amount_1);
       let second_opening = PedersenOpening::new_rand();
       let second_ciphertext = second_keypair
           .pubkey()
           .encrypt_with(amount_2, &second_opening);

       let mut prover_transcript = Transcript::new(b"Test");
       let mut verifier_transcript = Transcript::new(b"Test");

       // Call the `new` wrapper with inconsistent inputs:
       //    - `first_ciphertext` encrypts `amount_1`
       //    - `second_opening` used to encrypt `amount_2`
       // Try to claim the common amount is `amount_1`
       // The function should fail but it does not. It produces a proof object.
       let proof = CiphertextCiphertextEqualityProof::new(
           &first_keypair,
           second_keypair.pubkey(),
           &first_ciphertext,  // Encrypts 50
```

```
            &second_opening,    // Encrypts 100
            amount_1,           // Claiming the amount is 50
            &mut prover_transcript,
        );

        //  The resulting proof will
        //  always fail verification, wasting fees.
        assert!(proof.verify(
            first_keypair.pubkey(),
            second_keypair.pubkey(),
            &first_ciphertext,
            &second_ciphertext,
            &mut verifier_transcript
        ).is_err());
    }
```

File: `zk-sdk/src/zk_elgamal_proof_program/proof_data/ciphertext_ciphertext_equality.rs`

```
#[test]
    fn test_wrapper_accepts_mismatched_inputs() {
        // Setup: two keypairs and two distinct amounts
        let first_keypair = ElGamalKeypair::new_rand();
        let second_keypair = ElGamalKeypair::new_rand();

        let amount_1: u64 = 50;
        let amount_2: u64 = 100;

        // Create the ciphertexts corresponding to the two distinct amounts
        let first_ciphertext = first_keypair.pubkey().encrypt(amount_1);
        let second_opening = PedersenOpening::new_rand();
        let second_ciphertext = second_keypair
            .pubkey()
            .encrypt_with(amount_2, &second_opening);

        // Call the high-level `new` with inconsistent inputs:
        //    - `first_ciphertext` encrypts `amount_1`
        //    - `second_opening` used to encrypt `amount_2`
        // Try to claim the common amount is `amount_1`
        // The function should fail but it does not. It produces a proof object.
        let proof = CiphertextCiphertextEqualityProofData::new(
            &first_keypair,
            second_keypair.pubkey(),
            &first_ciphertext,   // Encrypts 50
            &second_ciphertext,  // Encrypts 100
            &second_opening,     // Used to encrypt 100
            amount_1,            // Claiming the amount is 50
        );
```

```
        assert!(proof.is_ok());

        //  The resulting proof will
        //  always fail verification, wasting fees.
        assert!(proof.unwrap().verify_proof().is_err());
    }
```

Recommendations:

The provided test cases demonstrate that both the core `CiphertextCiphertextEqualityProof::new` API and its high-level wrapper, `CiphertextCiphertextEqualityProofData::new`, are flawed as they fail to enforce their own cryptographic invariants. This violates the principle of "secure by default" and pushes the burden of validation onto the developer. The high-level `CiphertextCiphertextEqualityProofData::new` function must be modified to explicitly validate the consistency of its inputs *before* proceeding with proof generation.

Since analogous issues are present in all sigma_proofs and proof_data files implementing arguments of knowledge, all affected files should be updated accordingly.

Issue: Missing Identity Point Validation [#b14]

as discussed with the Anza team

Severity: Medium impact

Description:

Deserialization of `ElGamalPubkey`, `ElGamalCiphertext`, `PedersenCommitment`, and `DecryptHandle` does not reject the identity point (all zeros). More importantly, When identity points reach verification equations, some terms vanish. This affects all proof types using these primitives. Tests in this file demonstrate identity pubkeys are accepted and reach algebraic verification.

Code:

Repository: zk-elgamal-proof

File: `ciphertext_ciphertext_equality.rs`

```
/// Identity pubkey substitution fails at algebraic check, not at deserialization.
    #[test]
    fn test_identity_pubkey_soundness_full() {
        let first_keypair = ElGamalKeypair::new_rand();
        let second_keypair = ElGamalKeypair::new_rand();
        let second_opening = PedersenOpening::new_rand();

        let proof_data = CiphertextCiphertextEqualityProofData::new(
            &first_keypair,
            second_keypair.pubkey(),
            &first_keypair.pubkey().encrypt(42u64),
            &second_keypair.pubkey().encrypt_with(42u64, &second_opening),
            &second_opening,
            42u64,
```

```
        ).unwrap();

        // Substitute identity pubkey - should be rejected early
        let attack_data = CiphertextCiphertextEqualityProofData {
            context: CiphertextCiphertextEqualityProofContext {
                first_pubkey: PodElGamalPubkey([0u8; 32]),
                ..proof_data.context
            },
            proof: proof_data.proof,
        };

        assert!(format!("{:?}", attack_data.verify_proof()).contains("AlgebraicRel
ation"));
    }
```

Recommendations:

Apply to all deserialization paths: (as explained by the Anza team, not very practical and requires other changes to support the auditor flow of CT)

```
  let point =  compressed.decompress()
               .ok_or(ElGamalError::PubkeyDeserialization)?;
  reject_identity(&point)?;
  Ok(ElGamalPubkey(point))

// PedersenCommitment::from_bytes
  ...
```

OR

Make sure it is checked and rejected early in the `.verify()` path.

affected sigma protocols:

- ciphertext_ciphertext_equality.rs
- ciphertext_commitment_equality.rs
- zero_ciphertext.rs
- percentage_with_cap.rs
- grouped_ciphertext_validity/handles_2.rs
- grouped_ciphertext_validity/handles_3.rs

Issue: Identity Point Passes Verification in Range Proofs Stack [#b15]

Severity: Medium (High for future code updates and maintainability)

Description:

`RangeProof::new()` and `RangeProof::verify()` accept identity commitments without validation. The public `verify_proof()` API is only protected by an implicit check—identity commitment bytes `[0u8; 32]` match the empty slot and get filtered out by `take_while`. This is not an explicit security check. Internal code using `RangeProof` directly is vulnerable, and any future encoding change could expose the public API.

Code:

Repository: zk-elgamal-proof

File: `batched_range_proof_u64.rs`

```
/// The identity commitment has bytes [0u8; 32], which is filtered out by
/// try_into() take_while check. So verify_proof() is protected (by accident?).
/// RangeProof::verify() at the primitive level has no protection.
#[test]
fn range_proof_accepts_identity_commitment() {
    // Create identity commitment
    let zero_opening = PedersenOpening::default();
    let identity_commitment = Pedersen::with(0_u64, &zero_opening);
    assert!(identity_commitment.get_point().is_identity());

    // Identity commitment bytes are all zeros
    assert_eq!(identity_commitment.to_bytes(), [0u8; 32]);

    // Create context for transcript
    let mut commitments = [PodPedersenCommitment([0u8; 32]); 8];
    commitments[0] = PodPedersenCommitment(identity_commitment.to_bytes());
    let context = BatchedRangeProofContext {
        commitments,
        bit_lengths: [64, 0, 0, 0, 0, 0, 0, 0],
    };

    // RangeProof::new() accepts identity commitment - no check
    let mut prover_transcript = context.new_transcript();
    let proof = RangeProof::new(
        vec![0_u64],
        vec![64],
        vec![&zero_opening],
        &mut prover_transcript,
    );
    assert!(proof.is_ok(), "RangeProof::new() should accept identity");

    // RangeProof::verify() accepts identity commitment - no check
    let mut verifier_transcript = context.new_transcript();
    let proof = proof.unwrap();
    let result = proof.verify(
        vec![&identity_commitment],
        vec![64],
        &mut verifier_transcript,
```

```
    );
    assert!(result.is_ok(), "RangeProof::verify() should accept identity");

    // verify_proof() is protected
    // implicit check only
    let pod_proof: PodRangeProofU64 = proof.try_into().unwrap();
    let proof_data = BatchedRangeProofU64Data { context, proof: pod_proof };
    let result = proof_data.verify_proof();
    assert!(result.is_err(), "verify_proof() should reject (identity filtered as z
eroed)");
}
```

Recommendations:

Add explicit `is_identity()` check in `RangeProof::verify()` to reject identity commitments intentionally, rather than relying on an implicit side effect.

Issue: Insufficient Proof Data Account Size and Offset Validation [#b16]

as discussed with the Anza team

Severity: Medium

Description:

Proof data can be stored in an unnecessarily large account, and the program reads it from arbitrary offsets without enforcing an upper bound on the account size or the accessed slice.

Code:

Repository: zk-elgamal-proof

```
#[cfg(test)]
mod tests {
    use {
        bytemuck::bytes_of,
        solana_account::Account,
        solana_program_test::*,
        solana_pubkey::Pubkey,
        solana_signer::Signer,
        solana_transaction::Transaction,
        solana_zk_sdk::{
            encryption::elgamal::ElGamalKeypair,
            zk_elgamal_proof_program::{
                instruction::*, proof_data::*,
            },
        },
    };
```

```
    #[tokio::test]
    async fn test_excessive_proof_size_and_offset() {
        let mut program_test = ProgramTest::default();
        program_test.set_compute_max_units(500_000);

        let elgamal_keypair = ElGamalKeypair::new_rand();
        let zero_ciphertext = elgamal_keypair.pubkey().encrypt(0_u64);
        let proof_data =
            ZeroCiphertextProofData::new(&elgamal_keypair, &zero_ciphertext).unwra
p();

        // Create an unnecessary large account (10MB)
        let mut account_data = vec![0u8; 10 * 1024 * 1024];
        let proof_bytes = bytes_of(&proof_data);

        // Place proof at a very high offset
        let extreme_offset = 40_000_usize;
        account_data[extreme_offset..extreme_offset + proof_bytes.len()]
            .copy_from_slice(proof_bytes);

        let proof_account = Pubkey::new_unique();
        program_test.add_account(
            proof_account,
            Account {
                lamports: 1_000_000_000,
                data: account_data,
                owner: Pubkey::new_unique(),
                ..Account::default()
            },
        );
        let mut context = program_test.start_with_context().await;
        let client = &mut context.banks_client;
        let payer = &context.payer;


        let instruction = ProofInstruction::VerifyZeroCiphertext
            .encode_verify_proof_from_account(None, &proof_account, extreme_offset
as u32);

        let transaction = Transaction::new_signed_with_payer(
            &[instruction],
            Some(&payer.pubkey()),
            &[payer],
            client.get_latest_blockhash().await.unwrap(),
        );

        let result = client.process_transaction(transaction).await;
```

```
        assert!(result.is_ok());
    }
```

Recommendations:

Consider setting a hard limit on context-state account length: the actual maximum size across all proof types multiplied by the maximum number of proofs per account used in CT. Compute the expected length from the verified proof context and compare immediately. If the account length differs or exceeds the cap then return `InvalidAccountData`.
Perform these size checks before parsing any metadata, and before any writes. This forces fast failure and blocks oversized accounts without extra work.

Discussion:

Solana runtime still loads the account data into memory before the program
executes. The size checks only reject the transaction after the oversized account has already been loaded, so they reduce wasted compute but cannot prevent the initial memory consumption that enables the DoS vector. At the Solana protocol level, the validator pre-loads all accounts referenced by a transaction into validator memory before any program
logic runs. However, the cache mechanism makes this attack vector theoretical only due to high cost.

Future work: check economical viability and impact of:

- Modifying data of the allocated buffer after each transaction to force cache invalidation.
- Redeem rent for the allocated space and re-alocate to avoid the cache mechanism effect.

We stop the investigation at this point due to the fact that besides the concrete recommendations given, it is not a CT issue.

Issue: Test Suites Lack Negative and Adversarial Testing [#b17]

Severity: Low impact / best practices

Description:

The test suites for numerous core cryptographic components focus almost exclusively on the "happy path." They correctly confirm that a proof generated from valid inputs will pass verification (checking for *completeness*), but this is insufficient for security-critical code.

A robust test suite must also assert that the verifier rejects invalid inputs and malformed proofs (checking for *soundness*). The test suites does not cover these critical negative test cases, such as:

1. A valid sigma proof submitted with a mismatched public input (incorrect commitment or public key).
2. A sigma proof submitted with a public key or handle that is the identity point.
3. A range proof submitted with an incorrect range.

This issue applies to the test suites within at least the following files, but probably many more:

- `zk-sdk/src/range-proof/mod.rs`
- `zk-sdk/src/range-proof/inner_product.rs`

- `zk-sdk/src/zk_elgamal_proof_program/proof_data/ciphertext_ciphertext_equality.rs`
- `zk-sdk/src/zk_elgamal_proof_program/proof_data/grouped_ciphertext_validity/grouped_ciphertext_2_handles.rs`
- `zk-sdk/src/zk_elgamal_proof_program/proof_data/grouped_ciphertext_validity/grouped_ciphertext_3_handles.rs`
- `zk-sdk/src/zk_elgamal_proof_program/proof_data/batched_grouped_ciphertext_validity/batched_grouped_ciphertext_2_handles.rs`
- `zk-sdk/src/zk_elgamal_proof_program/proof_data/batched_grouped_ciphertext_validity/batched_grouped_ciphertext_3_handles.rs`
- `zk-sdk/src/sigma_proofs/grouped_ciphertext_validity/grouped_ciphertext_2_handles.rs`
- `zk-sdk/src/sigma_proofs/grouped_ciphertext_validity/grouped_ciphertext_3_handles.rs`
- `zk-sdk/src/sigma_proofs/batched_grouped_ciphertext_validity/batched_grouped_ciphertext_2_handles.rs`
- `zk-sdk/src/sigma_proofs/batched_grouped_ciphertext_validity/batched_grouped_ciphertext_3_handles.rs`

Code:

Repository: zk-elgamal-proof

File: `zk-sdk/src/zk_elgamal_proof_program/proof_data/grouped_ciphertext_validity/grouped_ciphertext_2_handles.rs`

```
#[test]
fn test_ciphertext_validity_proof_instruction_correctness() {
    ...

    // This test confirms the above valid proof verifies, but never
    // confirms that an invalid proof fails verification.
    let proof_data = GroupedCiphertext2HandlesValidityProofData::new(
        first_pubkey,
        second_pubkey,
        &grouped_ciphertext,
        amount,
        &opening,
    )
    .unwrap();

    assert!(proof_data.verify_proof().is_ok());
}
}
```

Recommendations:

All proof-related test suites should be augmented with comprehensive negative test cases. These tests should be adversarial, attempting to break or at least undermine of verifier's assumptions.

Issue: Documentation is not properly linked to the code [#b18]

Severity: Low impact / best practices

Description:

The protocol documentation PDFs referenced and used during the design and security analysis of the zk-ElGamal proof system are not included in the zk-elgamal-proof GitHub repository. Specifically, the following documents are missing from the repo:

- ciphertext_ciphertext_equality.pdf
- ciphertext_commitment_equality.pdf
- ciphertext_validity.pdf
- percentage_with_cap.pdf
- pubkey_proof.pdf
- twisted_elgamal.pdf
- zero_proof.pdf

These documents contain the formal specifications and security arguments for the various zero-knowledge proof components. Without them, external reviewers and future maintainers lack an authoritative specification to compare against the implementation, making independent security review, maintenance, and future extensions harder and more error-prone.

Code:

Repository: zk-elgamal-proof

Recommendations:

1. Add the listed PDF documents to the zk-elgamal-proof repository, under a clearly named directory such as `docs/`.
2. Reference these documents from the main `README` so that reviewers and developers can easily find the formal protocol specifications.
3. Ensure the PDFs are the most updated versions used to implement the current code.